

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Выполнил:

Грищук Елизавета Дмитриевна
БР 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

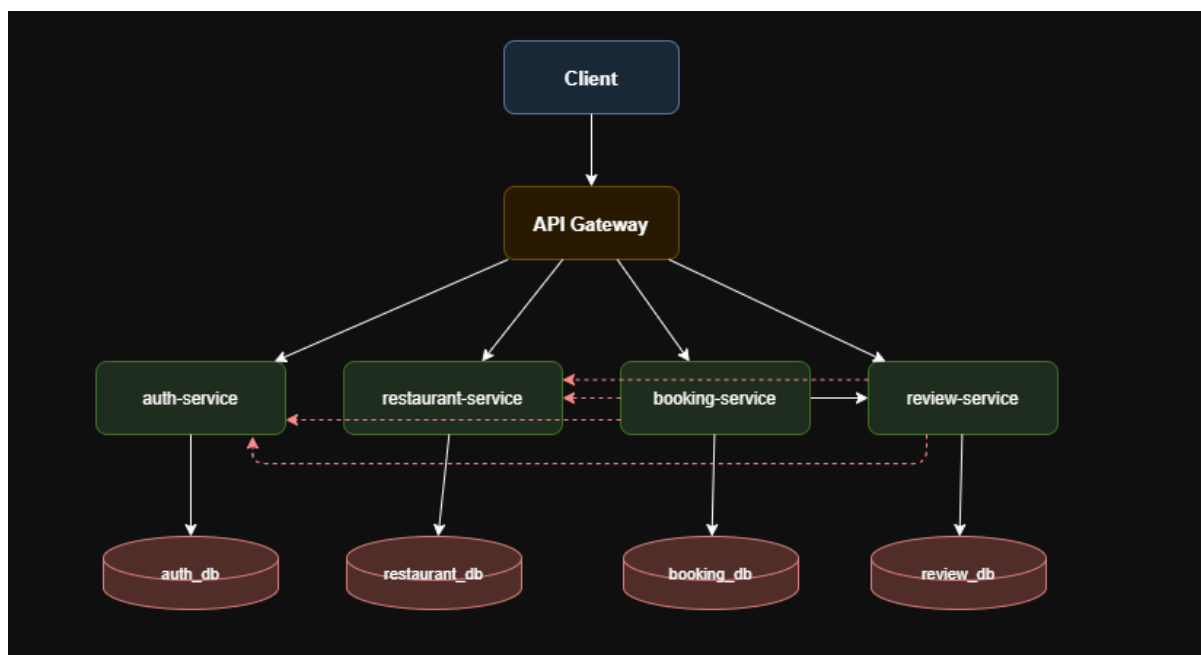
Разделить монолитное приложение бронирования столиков в ресторанах на микросервисы, придерживаясь концепции database-per-service. Спроектировать архитектуру, описать способы взаимодействия между сервисами, спроектировать разделение БД и описать внутренние эндпоинты в формате OpenAPI.

Ход работы

1. Разделение на микросервисы

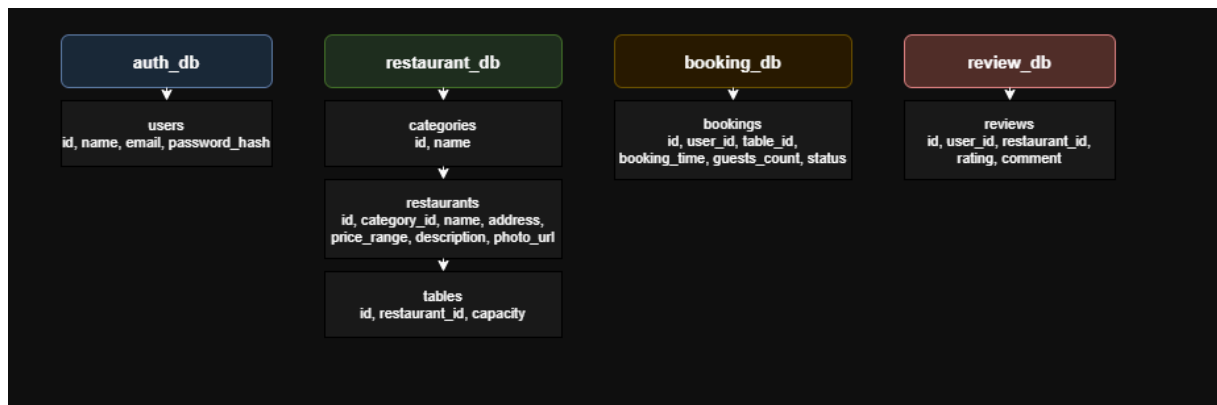
Монолитное приложение разделено на 4 микросервиса:

- auth-service (порт 3001) — регистрация и авторизация пользователей, управление профилем. База данных: auth_db, таблица users.
- restaurant-service (порт 3002) — управление категориями, ресторанами и столиками. База данных: restaurant_db, таблицы categories, restaurants, tables.
- booking-service (порт 3003) — создание, просмотр и отмена бронирований. База данных: booking_db, таблица bookings.
- review-service (порт 3004) — создание и просмотр отзывов. База данных: review_db, таблица reviews.



2. Разделение базы данных

В соответствии с принципом database-per-service каждый микросервис имеет собственную изолированную базу данных. Прямые JOIN между базами данных разных сервисов исключены — данные получаются через межсервисные HTTP-запросы.



3. Способы взаимодействия

Клиент взаимодействует с системой через API Gateway, который маршрутизирует запросы к нужному сервису. Межсервисное взаимодействие реализовано через синхронные HTTP-запросы (REST) по внутренней сети:

- booking-service → auth-service: перед созданием бронирования проверяет существование пользователя через GET /internal/users/{id}
- booking-service → restaurant-service: проверяет существование столика и его вместимость через GET /internal/tables/{id}
- review-service → auth-service: перед созданием отзыва проверяет существование пользователя через GET /internal/users/{id}
- review-service → restaurant-service: проверяет существование ресторана через GET /internal/restaurants/{id}

4. Внутренние эндпоинты (межсервисное взаимодействие)

Спроектированы следующие внутренние эндпоинты, недоступные клиенту напрямую:

auth-service:

- GET /internal/users/{id} — получить пользователя по id. Возвращает id, name, email. Ошибки: 404.

restaurant-service:

- GET /internal/tables/{id} — получить столик по id с вместимостью. Ошибки: 404.
- GET /internal/restaurants/{id} — получить ресторан по id. Ошибки: 404.

Полная спецификация внутренних эндпоинтов реализована в формате OpenAPI 3.0 (файл internal-api.yaml).

5. План миграции с монолита на микросервисы

Шаг 1 — Разделение базы данных

Из единой базы restaurant_booking создать 4 отдельные базы:

- auth_db — перенести таблицу users
- restaurant_db — перенести таблицы categories, restaurants, tables
- booking_db — перенести таблицу bookings
- review_db — перенести таблицу reviews

Внешние ключи между таблицами разных баз удалить — целостность данных обеспечивается на уровне приложения через межсервисные запросы.

Шаг 2 — Выделение auth-service

Из монолита выделить всю логику, связанную с пользователями: регистрация, авторизация, JWT, профиль. Сервис подключается только к auth_db. Добавить внутренний эндпоинт GET /internal/users/{id} для других сервисов.

Шаг 3 — Выделение restaurant-service

Выделить логику категорий, ресторанов и столиков. Сервис подключается только к restaurant_db. Добавить внутренние эндпоинты GET /internal/tables/{id} и GET /internal/restaurants/{id}.

Шаг 4 — Выделение booking-service

Выделить логику бронирований. Сервис подключается только к booking_db. При создании бронирования сервис делает HTTP-запросы к auth-service и restaurant-service для проверки пользователя и столика.

Шаг 5 — Выделение review-service

Выделить логику отзывов. Сервис подключается только к review_db. При создании отзыва делает HTTP-запросы к auth-service и restaurant-service.

Шаг 6 — Настройка API Gateway

Настроить API Gateway как единую точку входа для клиента. Gateway маршрутизирует запросы по префиксу пути:

Префикс	Сервис
/auth/*	auth-service:3001
/users/*	auth-service:3001
/categories/ /restaurants/	restaurant-service:3002
/bookings/*	booking-service:3003
/reviews/*	review-service:3004

Шаг 7 — Проверка работоспособности

После миграции прогнать коллекцию Postman из Д33 — все 10 запросов должны работать так же как в монолите.

Вывод

В ходе выполнения работы монолитное приложение было декомпозировано на 4 микросервиса: auth-service, restaurant-service, booking-service, review-service. Каждый сервис имеет собственную базу данных в соответствии с принципом database-per-service. Спроектирована архитектура взаимодействия через API Gateway и внутренние HTTP-запросы между сервисами. Описаны 3 внутренних эндпоинта для межсервисной коммуникации в формате OpenAPI 3.0.